

Tether Game-Feel Post-Mortem

How "it's slippery and not responsive" took 33 builds to fix — every mistake, how it was found, and what finally worked. [lifegameproject.com · game/rope.html](https://lifegameproject.com/game/rope.html) · builds b1–b33 · July 2026

The problem

Tether is a two-player co-op rope-climbing game (Verlet physics, fixed 60 Hz simulation, HTML5 canvas). From its first playable build the player consistently reported that left/right movement felt **slippery** (kept moving after input stopped), **unresponsive** (didn't move when input started), and **inconsistent** ("the same tap goes a little or a lot"). It took 33 tagged builds to fully resolve — not because any single fix was hard, but because **multiple independent defects produced the same symptom**, masking each other, and several of my own fixes introduced new versions of the problem.

The mistakes, in order

1 · Tuning numbers instead of measuring behavior

WHAT I DID Responded to "slippery" by adjusting friction/speed constants and shipping, repeatedly.

WHY WRONG The slipperiness was caused by structural bugs (see below), so no constant could fix it. Several rounds of effort produced zero improvement and eroded trust.

CORRECTION Stopped tuning; built measurement rigs: a Node physics simulator driving the game's real `step()` function, and real-browser telemetry (inject synthetic KeyboardEvents, sample position every frame, POST the data out).

Lesson: a feel complaint is a measurement request. Convert adjectives to numbers before changing anything.

2 · Phantom footing (missed pre-existing bug)

WHAT HAPPENED Rising up through a one-way platform counted as "standing" on it for a few frames. Mid-jump the game granted stray footing: phantom jumps, ground-brakes chopping speed mid-air, moving platforms dragging the ball sideways while passing through.

FOUND BY A frame-by-frame trace of a jump — "grounded" flickered true at flight apex frame 4.

CORRECTION `platformUnder()` returns nothing while moving upward fast (a rising body is never standing).

Lesson: when a symptom survives an input-system rewrite, the bug is probably in the physics the input drives.

3 · The backwards force ramp

WHAT I DID Made *force* build up with key-hold time ("hold longer = push harder"), starting at 35% strength.

WHY WRONG Starts and direction reversals — exactly where responsiveness is felt — were the weakest moments. Turning around took ~270 ms of mush.

CORRECTION Full force from frame one; hold-time may shape *top speed*, never initial response. Reversals got a turn-assist that chops opposing velocity.

Lesson: ramp outputs (speed), never inputs (response). The first frame after a key press is sacred.

4 · Eaten jumps — no buffering, no coyote time

WHAT HAPPENED Jump pressed one frame before landing → consumed mid-air, nothing happens. Stepping off a ledge and pressing a moment late → nothing.

CORRECTION 9-frame jump buffer (early presses fire on touchdown) + 7-frame coyote window (late presses still work just after leaving a ledge) — standard platformer forgiveness I should have built from day one.

Lesson: buffers belong to one-shot *actions*. Every platformer needs jump buffering + coyote time; it is not optional polish.

5 · Frame aliasing, then my "fix" overshoot

WHAT HAPPENED A ~60 ms click is sampled as 1–4 sim frames depending on phase → identical taps moved 1–4× different distances. I added a minimum force pulse (good), but let pulses **stack** — three drummed taps forced ~58 px of movement *after* the last release.

FOUND BY An arithmetic audit tracing per-frame velocity for tap sequences.

CORRECTION Pulses never stack; releases truncate them. (Later versions restructured this entirely — see #9/#11.)

Lesson: every forgiveness mechanism is also an unintended-motion generator. Bound it, then test its worst case.

6 · The taut-rope brake exemption + an invalid test

WHAT I DID Exempted a player from braking while the rope was taut ("being dragged should be smooth"). In solo play the rope is taut almost always → both balls skated ~23 px after every release. My validation missed it because the test ball was pinned against the arena wall when I measured.

CORRECTION Reverted after an A/B simulation in open field: 22.6 px slide with the exemption, 4.7 px without — and dragging worked fine anyway (the rope constraint is positional; it doesn't need the brake gone).

Lesson: validate in the scenario the player actually plays. A measurement at a boundary (wall, edge) can silently measure nothing.

7 · Testing at 60 Hz for a 120 Hz player

WHAT HAPPENED The sim steps at 60 Hz; the player's ProMotion display runs 120 Hz. Every second screen frame duplicated the previous state — motion judder that reads as "floaty/glitchy". My headless test rig ran at 60 Hz, where this *cannot appear*, so I kept "proving" smoothness the player couldn't feel.

FOUND BY The player's screenshot showing "b15 · 120fps" in the on-page fps meter I'd added.

CORRECTION Fixed-timestep render interpolation (later: extrapolation) so high-refresh displays get unique frames.

Lesson: know the player's hardware. Put build + fps on screen — it converts "works for me" arguments into data.

8 · Interpolation's hidden frame of latency

WHAT HAPPENED Classic interpolation renders *between the previous and current* physics states — buttery, but up to one frame behind. The player felt "slow to act after I click".

CORRECTION Extrapolate the player and rope (project the newest state forward by the frame fraction) while the camera/world stay interpolated. Later, the tap's first pixels were moved into the keydown handler itself — visible at the next screen refresh (~8 ms at 120 Hz). Measured click-to-pixels: 11.5–12.3 ms.

Lesson: smoothness techniques buy latency. For player-controlled objects, prefer extrapolation; artifacts can be masked with animation (squash/stretch).

9 · Movement as commands — the architectural dead end (b19–b25)

WHAT I DID Built "tap-to-step": each tap a queued step, then chaining, then cadence bridging, then opposite-tap queues, then anti-collision trims — seven builds of patches.

WHY WRONG Commands serialize. Every patch moved the wait somewhere else: "waiting for the previous one to finish", steps eaten by queue caps, lag that *accumulated* after three rounds of alternation (bridged step duration = tap gap $\Rightarrow$ guaranteed collisions with human timing jitter). The player diagnosed it verbatim: "*almost like you are waiting for the previous one to finish.*"

CORRECTION Threw the architecture away twice — first for polled state (#10), finally for interrupt-driven steps (#11).

Lesson: if a design needs a queue for basic movement, the design is wrong. Also: when several consecutive fixes each "move" a symptom, stop patching — the architecture is the bug.

10 · Believing "instant full speed = precision"

WHAT HAPPENED With movement at full speed from frame one, a casual 100 ms tap flew a whole ball-width — "1 tap much further away". Published game-feel literature calls instant-acceleration-for-precision a myth (Celeste ramps to max over ~6 frames; stops are 2× faster than starts).

CORRECTION Implemented the researched curve — which improved taps but *still* didn't match this player's taste, leading to the real answer:

Lesson: research narrows the search space, but convention is a prior, not a verdict. The player's taste is the ground truth.

11 · Under-weighting the player's own spec

WHAT HAPPENED After 30 builds, the player wrote the design themselves: tap-only (hold does nothing), small fixed hop, hard stop, and — against every grid-mover convention — a new tap must **interrupt** the current one instantly. Implemented exactly that (b30/b31: instant 4.3 px nudge inside the keydown handler + one force frame + hard stop = 8.6 px hop, 0.00 px slide, overrides tracked 8/8 at every cadence). Verdict: "great it works much better now!"

Lesson: a frustrated player giving you a precise repro ("L1-R1 fine, L2-R2 lags") or a precise spec is handing you the answer. Their exact words beat ten diagnostic builds.

12 · The rope's velocity injection — the invariant everyone stepped over

WHAT HAPPENED In Verlet physics, every rope-constraint position correction becomes next-frame velocity. Each yank left a grounded player sliding and creeping like ice. Because it wasn't input, it survived every control-scheme rewrite — a big reason "slippery" kept coming back. An audit had flagged it early; I deferred it.

CORRECTION Grounded bodies take rope displacement as position only (px shifted alongside x): dragged exactly as far as the rope pulls, then stop dead. Post-release slide: 20–30 px → 1.4 px. Airborne untouched — there the injection *is* the pendulum swing.

Lesson: triage audit findings by "does this explain the user's words?", not by how tidy the fix is. And: the player never plays your isolated test scene — they play the coupled system.

13 · Shipping into a cache

WHAT HAPPENED GitHub Pages caches HTML ~10 minutes; builds shipped minutes apart. The player repeatedly feel-tested *stale builds*, making feedback uncorrelatable with changes. (Bonus trap: the iOS app bundles a frozen copy of the site — untestable-forever without a re-sync.)

CORRECTION On-page build tag ("b31 · 120fps"), version-stamped links (?v=b31) so the CDN can't serve yesterday's file, and a rule: never discuss feel without confirming the tag.

Lesson: version visibility is part of the product. If you can't prove which build the tester ran, you have no data.

What finally worked — the toolkit

- **1 Node physics simulation** — extract the game script, stub the DOM, drive `step()` directly; assert distances/frames deterministically.
- **2 Real-browser telemetry** — copy of the page + injected probe dispatching genuine KeyboardEvents, sampling per-rAF, POSTing results to a local server. Caught what Node couldn't (event pipeline, 120 Hz cadence).
- **3 A/B simulation** — old vs new behavior in the same scenario, away from boundaries.
- **4 Parallel forensic audits** — independent reviewers on physics and input pipeline; they found the piano-roll SOCD bug, macOS Cmd-swallowed keyups, air-model asymmetry, and the pulse-stacking arithmetic.
- **5 Build tag + fps on screen** — ended the stale-build confusion permanently.
- **6 Regression battery re-run after every feature** (orbs, swarm, toggles): the approved numbers must come back identical.

The final model (b31+, locked)

Tap-hop with instant override. Keydown → instant 4.3 px position nudge (inside the event handler) → one full-speed force frame → hard stop. Hop = 8.6 px, slide = 0.00 px. Any new tap interrupts the running step the same frame. Holding a key adds nothing. Jump keeps buffer + coyote (+ radiant mid-air jump). Physics persists only where it is the game: rope swings, jump arcs. Touch joystick stays continuous (positional = no ambiguity).

Measured: click→pixels ≈ 8–12 ms · direction flips tracked 8/8 at 33–200 ms tap cadences · 500 ms hold = one hop · drumming = continuous travel · all preserved bit-identical through the surge-swarm, hero-card, and power-up-orb additions (b32–b33).

Build timeline (abridged)

BUILD	CHANGE	PLAYER VERDICT
b1–b11	Tuning rounds; pause/anti-stall; audits begin	"still slippery"
b12–b13	Taut-brake exemption shipped, then reverted after A/B	"the same slippery again"
b14	11 audit fixes: SOCD last-press, Cmd keyups, jump buffer+coyote, air rework...	"omfg seriously"
b15–b16	fps meter + versioned links; direct control + 120 Hz interpolation	"it did feel better but..."
b17–b18	Rope strain visual, squash/stretch; airborne branch rebuilt (forensics)	"much better, but..."
b19–b25	The tap-command era: steps, chains, bridges, queues, trims	"waiting for the previous one"
b26–b27	Polled held state (industry standard); smaller nudge floor	"still slippery..."
b28	Rope velocity-injection fix + no-tether practice toggle	"1 feels bad" (no-tether)
b29	Celeste-calibrated acceleration curve (research)	closer, not it
b30–b31	The player's spec: tap-hop, instant override, keystroke-time nudge, no-surge toggle	"great it works much better now!"
b32–b33	Surge → color swarm; hero card; rope orbs + radiance — regressions all green	"its fine good job thanks"

Transferable lessons (the short list)

- **1** Measure before tuning. A feel adjective is a request for instrumentation.
- **2** Never buffer or queue basic movement. Buffers are for one-shot actions.
- **3** The first frame after input is sacred; ramp speed, never response.

- **4** Test on the player's refresh rate; show build + fps on screen.
- **5** Smoothness (interpolation) costs latency — extrapolate the player's own body.
- **6** Validate in realistic scenarios, away from walls and edges.
- **7** When fixes keep relocating a symptom, the architecture is the bug.
- **8** Coupled physics (the rope) can fake input problems; isolate with toggles (no-tether mode).
- **9** Research sets priors; the player's exact words are the evidence that outweighs them.
- **10** After every new feature, re-run the approved-feel regression and demand identical numbers.